12th Assignment

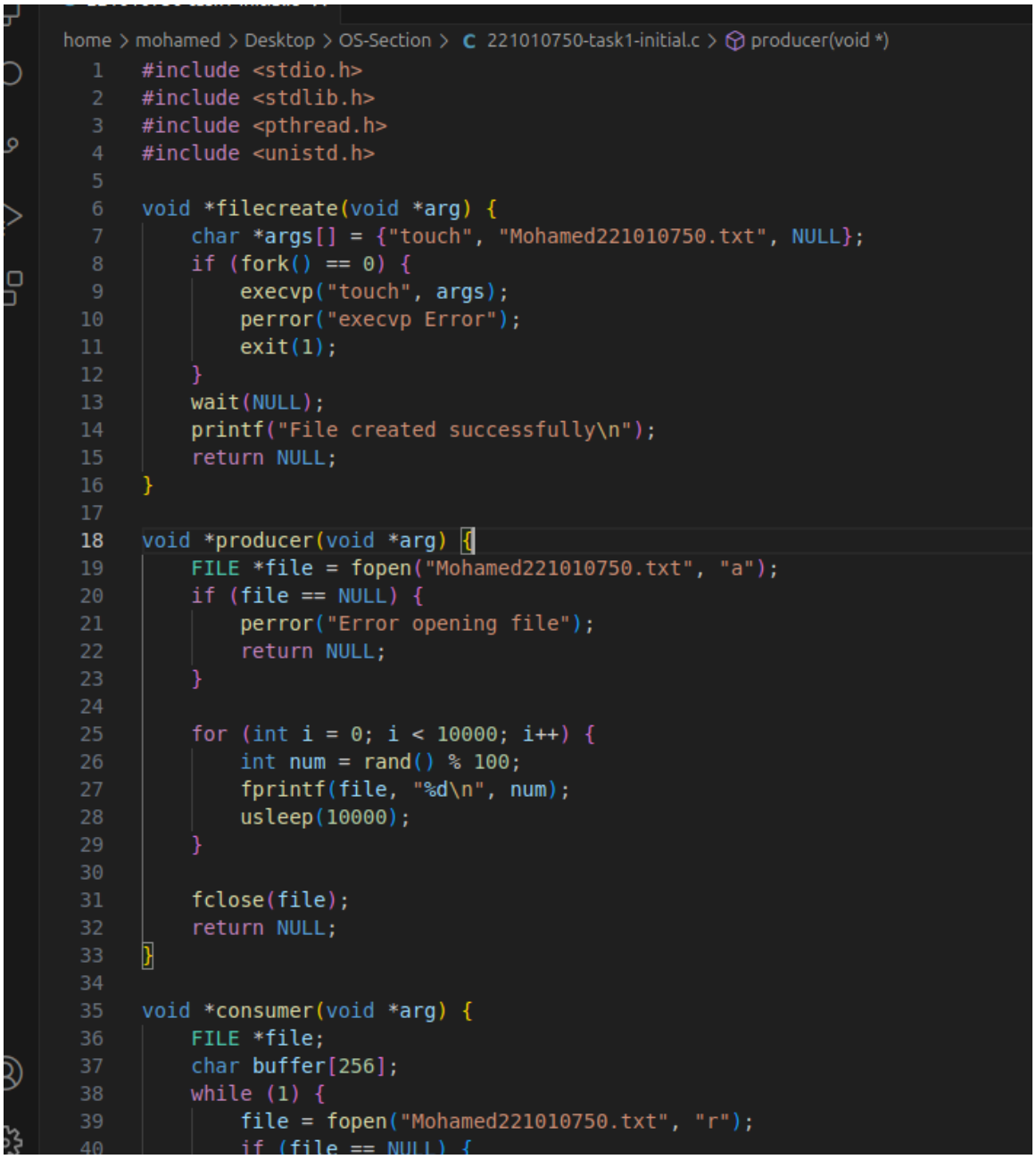
T.A: Abdelrhman Solyman

Name: Mohamed Elsayed Mohamed Khayami

Reg#: 221010750

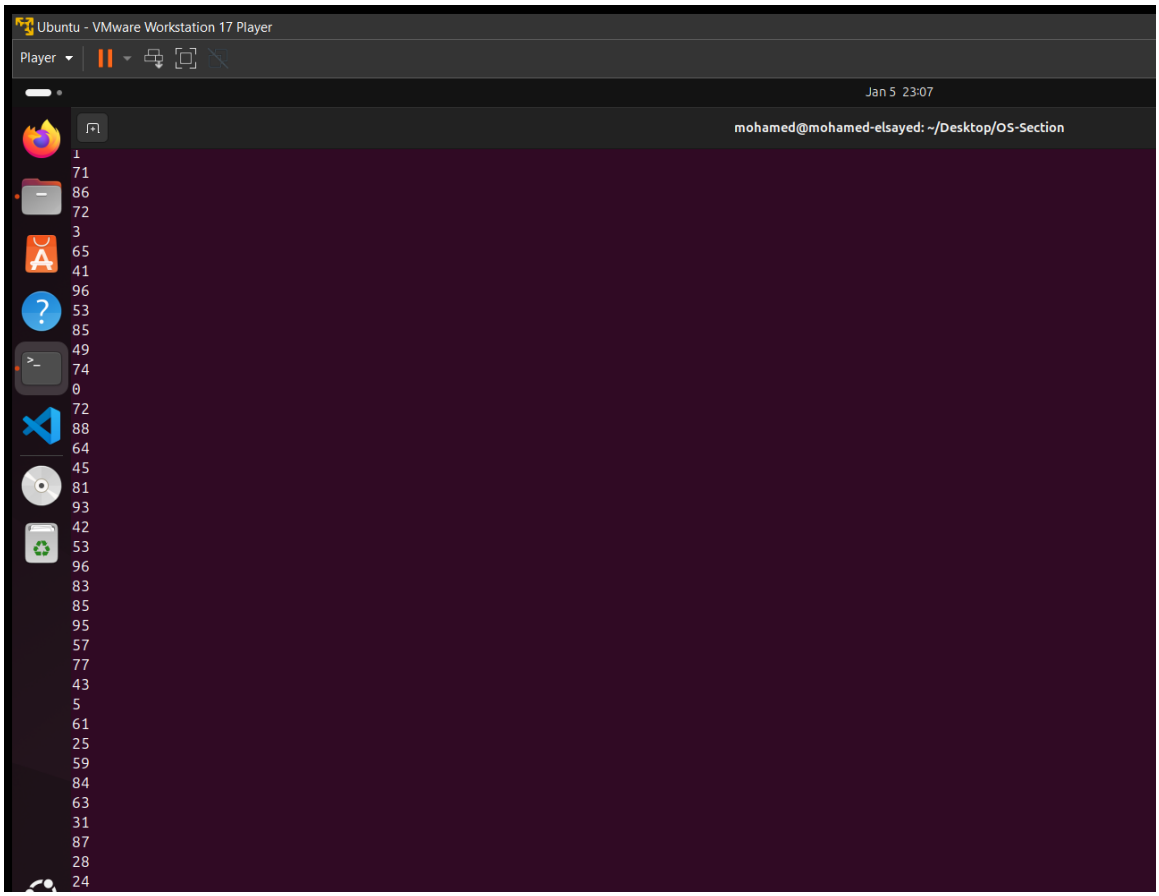
Task1:

c. (inserting integers into the files should be done concurrently)(attach Screenshot).



```
39          file = fopen("Mohamed221010750.txt", "r");
40          if (file == NULL) {
41              perror("Error opening file");
42              return NULL;
43          }
44
45          while (fgets(buffer, sizeof(buffer), file)) {
46              printf("%s", buffer);
47          }
48
49          fclose(file);
50          usleep(50000);
51      }
52      return NULL;
53  }
54
55  int main() {
56      pthread_t thread1, thread2, thread3;
57
58      pthread_create(&thread1, NULL, filecreate, NULL);
59      pthread_join(thread1, NULL);
60
61      pthread_create(&thread2, NULL, producer, NULL);
62      pthread_create(&thread3, NULL, consumer, NULL);
63
64      pthread_join(thread2, NULL);
65      pthread_cancel(thread3);
66
67      return 0;
68  }
69
```

d. 3rd Thread will act as consumer and read the files concurrently, print the output to console (attach Screenshot).

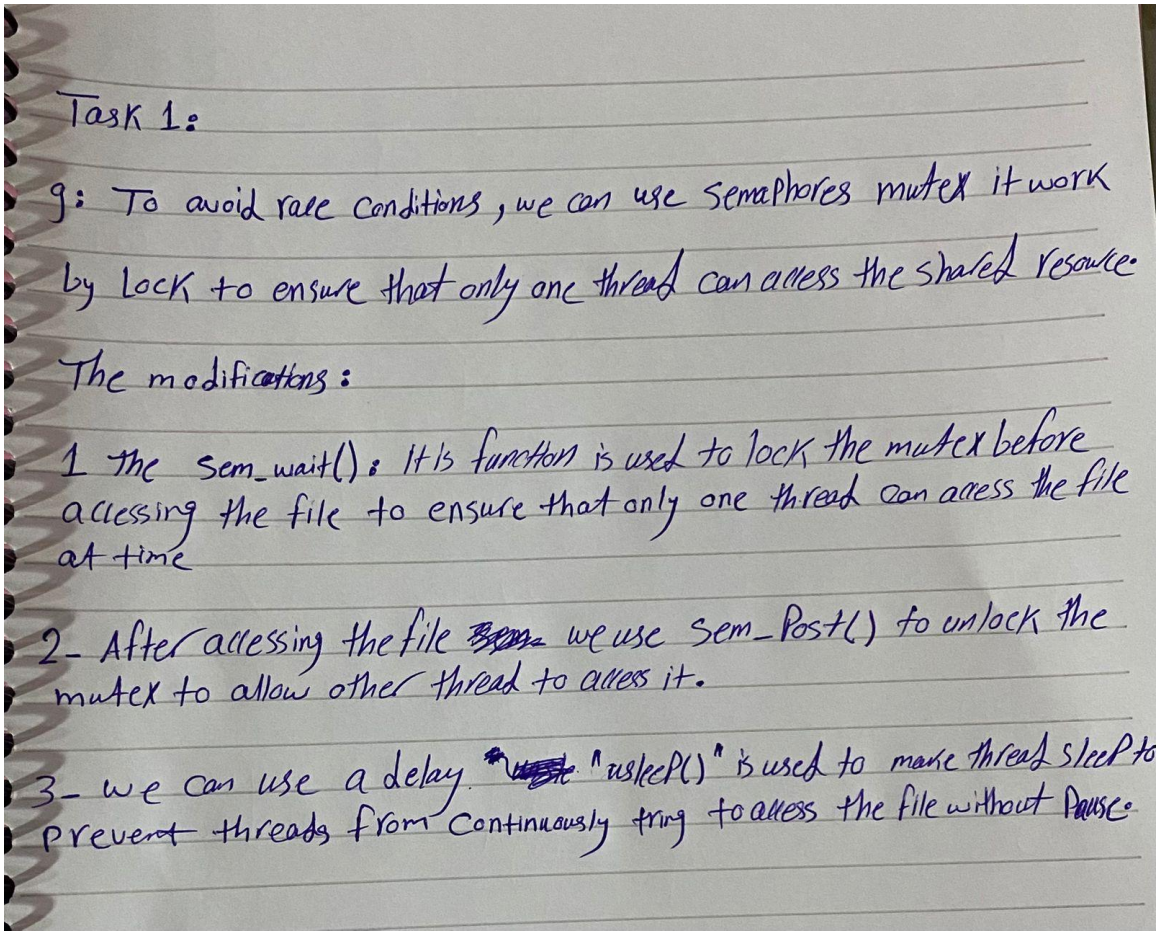


f. Identify the critical sections in the code (attach Screenshot).

```c
void *producer(void *arg) {
    FILE *file = fopen("Mohamed221010750.txt", "a");
    if (file == NULL) {
        perror("Error opening file");
        return NULL;
    }

    for (int i = 0; i < 10000; i++) {
        int num = rand() % 100;
        fprintf(file, "%d\n", num);
        usleep(10000);
    }

    fclose(file);
    return NULL;
}
```

```c
void *consumer(void *arg) {
    FILE *file;
    char buffer[256];
    while (1) {
        file = fopen("Mohamed221010750.txt", "r");
        if (file == NULL) {
            perror("Error opening file");
            return NULL;
        }

        while (fgets(buffer, sizeof(buffer), file)) {
            printf("%s", buffer);
        }

        fclose(file);
        usleep(10000);
    }
    return NULL;
}
```

g. Suggest your modifications to avoid the problems you may encounter “Racecondition” using semaphores to apply mutex, justify your answers in a written paper(attach scan copy of the paper).



Task 1:

g: To avoid race conditions, we can use semaphores mutex it work by Lock to ensure that only one thread can access the shared resource.

The modifications:

1 the sem_wait(): It is function is used to lock the mutex before accessing the file to ensure that only one thread can access the file at time

2- After accessing the file we use sem_Post() to unlock the mutex to allow other thread to access it.

3- we can use a delay "usleep()" is used to make thread sleep to prevent threads from continuously trying to access the file without Pause.

Task 2 [Bonus]:

c. (inserting integers into the files should be done concurrently with the consumer) (attach Screenshot).

221010750-bonus-initial.c

home > mohamed > Desktop > OS-Section > Bonus > 221010750-bonus-initial.c > thread_3(void *)

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <time.h>

#define MAX_RANDOM_INTS 100
#define MAX_LINES 3

char* filenames[2];

void* thread_1(void* arg);
void* thread_2(void* arg);
void* thread_3(void* arg);

int main() {
    pthread_t t1, t2, t3;

    pthread_create(&t1, NULL, thread_1, NULL);
    pthread_create(&t2, NULL, thread_2, NULL);
    pthread_create(&t3, NULL, thread_3, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);

    return 0;
}

void* thread_1(void* arg) {
    filenames[0] = "MohamedFile1.txt";
    filenames[1] = "MohamedFile2.txt";
    printf("Thread 1: Files created: %s, %s\n", filenames[0], filenames[1]);

    return NULL;
}

void* thread_2(void* arg) {
    char* file1 = filenames[0];
    char* file2 = filenames[1];

```

221010750-bonus-initial.c

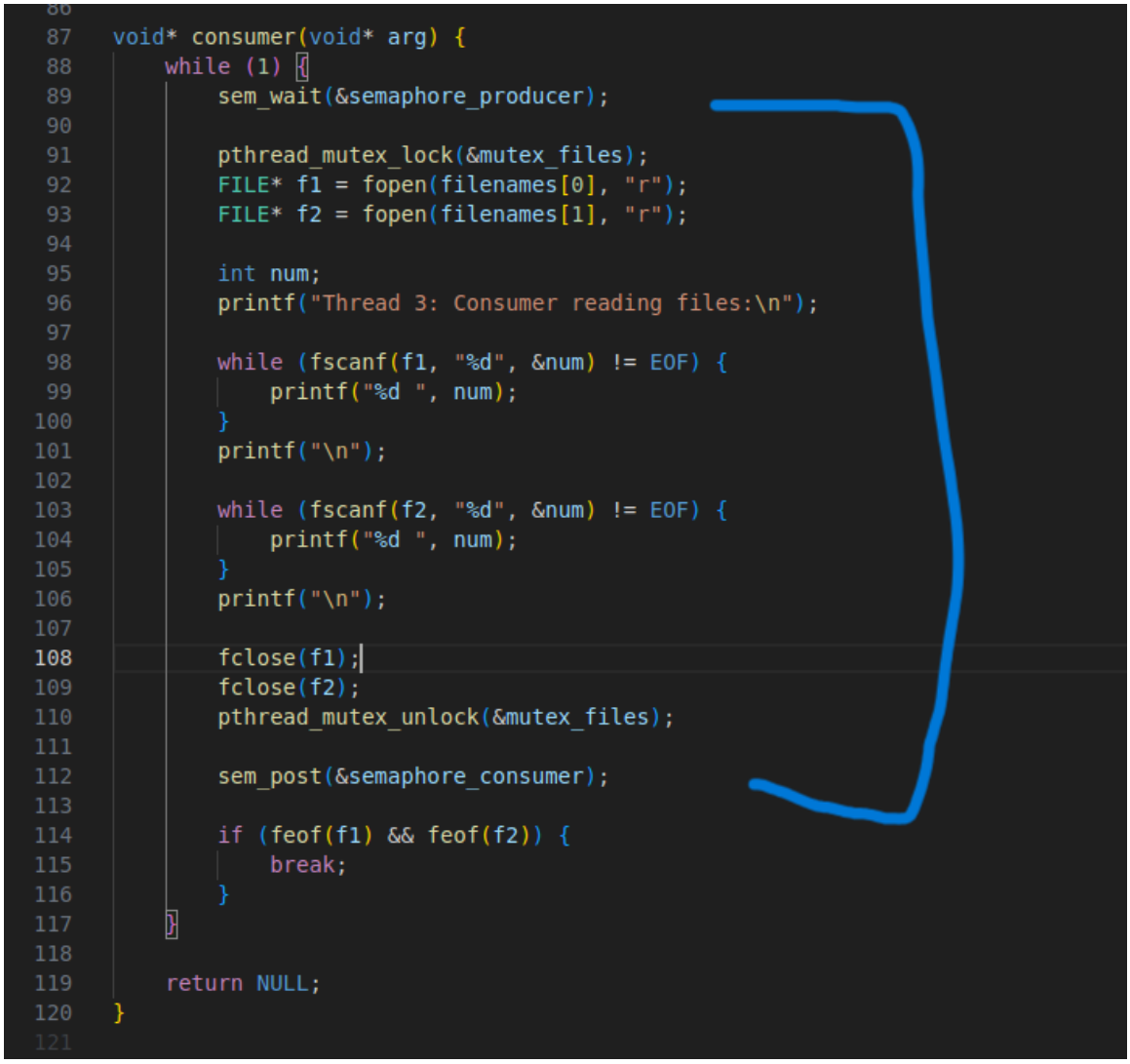
home > mohamed > Desktop > OS-Section > Bonus > 221010750-bonus-initial.c > thread_3(void *)

```c
void* thread_2(void* arg) {
    FILE* f2 = fopen(file2, "w");

    srand(time(NULL));
    for (int i = 0; i < MAX_LINES; i++) {
        int random_int = rand() % MAX_RANDOM_INTS;

        fprintf(f1, "%d\n", random_int);
        fprintf(f2, "%d\n", random_int);
    }

    fclose(f1);
    fclose(f2);

    return NULL;
}

void* thread_3(void* arg) {
    FILE* f1 = fopen(filenames[0], "r");
    FILE* f2 = fopen(filenames[1], "r");

    int num;
    printf("Thread 3: Consumer reading files:\n");

    while (fscanf(f1, "%d", &num) != EOF) {
        printf("%d ", num);
    }
    printf("\n");

    while (fscanf(f2, "%d", &num) != EOF) {
        printf("%d ", num);
    }
    printf("\n");

    fclose(f1);
    fclose(f2);

    return NULL;
}
```

d. 3rd Thread will act as consumer and read the files concurrently with the producer, print the output to console (attach Screenshot).

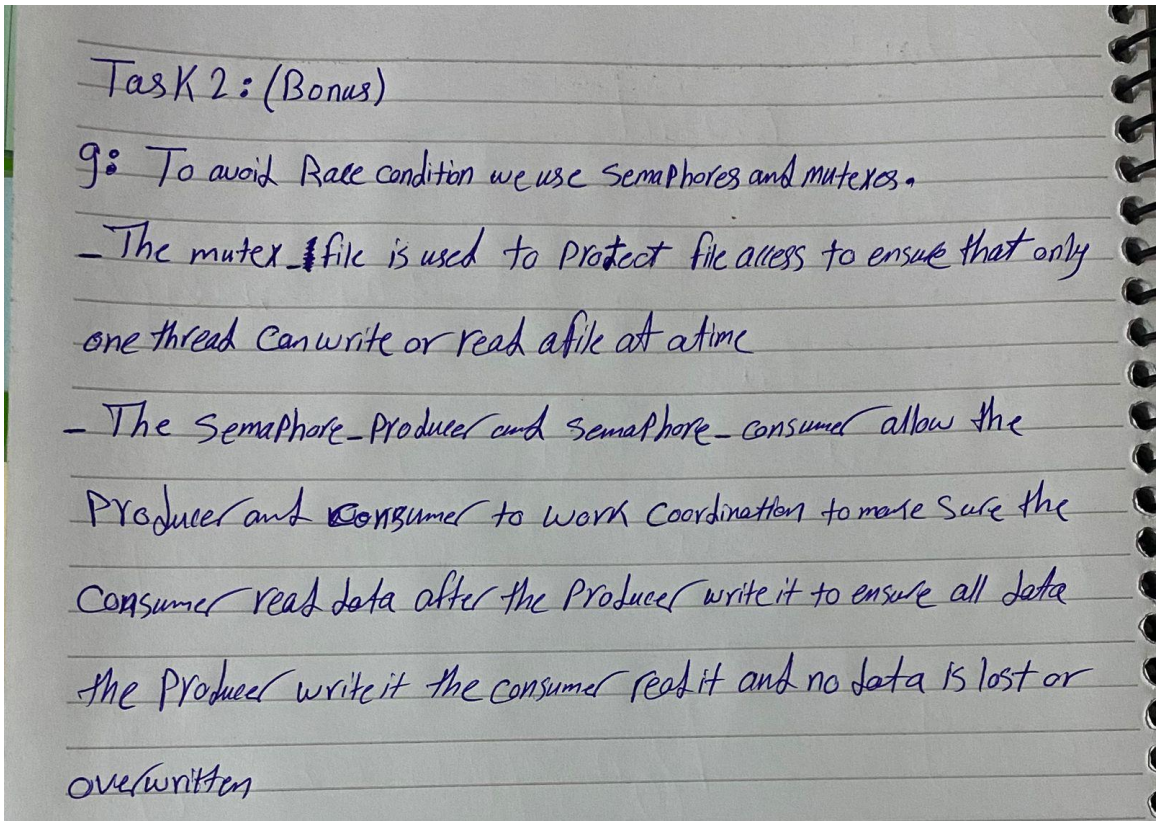
f. Identify the critical sections in the code (attach Screenshot).\

```c
void* producer(void* arg) {
    sem_wait(&semaphore_files);

    char* file1 = filenames[0];
    char* file2 = filenames[1];

    sem_post(&semaphore_files);

    FILE* f1 = fopen(file1, "w");
    FILE* f2 = fopen(file2, "w");

    srand(time(NULL));
    for (int i = 0; i < MAX_RANDOM_INTS; i++) {
        int random_int = rand() % 100;

        pthread_mutex_lock(&mutex_files);
        fprintf(f1, "%d\n", random_int);
        fprintf(f2, "%d\n", random_int);
        pthread_mutex_unlock(&mutex_files);

        sem_post(&semaphore_producer);
        sem_wait(&semaphore_consumer);
    }

    fclose(f1);
    fclose(f2);

    return NULL;
}

void* consumer(void* arg) {
```



```c
void* consumer(void* arg) {
    while (1) {
        sem_wait(&semaphore_producer);

        pthread_mutex_lock(&mutex_files);
        FILE* f1 = fopen(filenames[0], "r");
        FILE* f2 = fopen(filenames[1], "r");

        int num;
        printf("Thread 3: Consumer reading files:\n");

        while (fscanf(f1, "%d", &num) != EOF) {
            printf("%d ", num);
        }
        printf("\n");

        while (fscanf(f2, "%d", &num) != EOF) {
            printf("%d ", num);
        }
        printf("\n");

        fclose(f1);
        fclose(f2);
        pthread_mutex_unlock(&mutex_files);

        sem_post(&semaphore_consumer);

        if (feof(f1) && feof(f2)) {
            break;
        }
    }

    return NULL;
}
```

g. Suggest your modifications to avoid the problems you may encounter "Race condition" using semaphores to apply mutex among all threads, also state how can you check if all data produced by producer is read by consumer and no data was lost or overwritten, justify your answers in a written paper (attach scan of the paper).



Task 2: (Bonus)

g: To avoid Race condition we use Semaphores and mutexes.

- The mutex_file is used to protect file access to ensure that only one thread can write or read a file at a time
- The Semaphore_Producer and Semaphore_consumer allow the Producer and Consumer to work Coordination to make Sure the Consumer read data after the Producer write it to ensure all data the Producer write it the consumer read it and no data is lost or overwritten